Making Sense of DPU Performance for Cloud Data Processing: [Experiment, Analysis & Benchmark]

Jiasheng Hu, Chihan Cui, Yuanfan Chen, Philip A. Bernstein[†], Jialin Li[§], Qizhen Zhang

University of Toronto, [†]Microsoft Research, [§]National University of Singapore

ABSTRACT

Data processing units (DPUs, SoC-based SmartNICs) are emerging data center hardware that provide opportunities to address cloud data processing challenges. Their onboard compute, memory, network, and auxiliary storage can be leveraged to offload a variety of data processing tasks. Although recent work shows promising performance improvement and cost reduction by offloading specific operations to DPUs, further investigations into DPU offloading for databases can benefit from more detailed benchmarking.

This paper presents our efforts and results in benchmarking recent mass production DPUs to understand their performance characteristics for data processing tasks. A major challenge in benchmarking DPUs is the variety of DPUs from different generations and vendors. To handle the variety, we developed a DPU benchmarking framework that encompasses a suite of data processing tasks from primitive compute, memory, and I/O operations and hardware-accelerated tasks to macro-level cloud database modules and a full-fledged, lightweight DBMS. We provide an in-depth analysis of the results, which have uncovered novel insights into the performance of different DPU resources and different DPUs and their implications on cloud data processing.

1 INTRODUCTION

Databases [9, 16, 28, 61], KV stores [17, 24, 46, 57], and other data processing workloadshosted in cloud data centers are growing. So are the challenges. While computing demand for data processing keeps increasing due to data growth, general-purpose processors (CPUs) cannot sustain high compute efficiency due to the slowdown of hardware scaling laws. Hardware resource disaggregation of storage [21, 37, 62] and memory [36, 63, 69, 70] also decouples the software components of cloud-native systems, leading to intensive network communication, which has become the main performance bottleneck [61]. In addition, the speeds of storage and network devices are rapidly increasing; e.g., NVIDIA ConnectX-7 NICs provide 400 Gbps bandwidth [49]. Since the CPU instructions consumed to perform one byte of I/O are fixed, increases in disk and network bandwidth lead to higher CPU expenses [18, 32].

Data processing units (DPUs) are SoC-based SmartNICs that have emerged as popular programmable devices for in-network offloading. DPUs are characterized by a set of hardware resources designed to optimize data-path efficiency and offer simple programming interfaces. They are promising solutions to the above challenges in cloud data processing: CPU-intensive tasks (for both computational operators and I/O) can be offloaded to hardware accelerators and DPU cores to improve compute efficiency while reducing host CPU utilization; high-speed network connectivity and optimized I/O implementations can be leveraged to achieve better data movement performance over the network.

Current DPUs are designed to offload low-level packet processing, storage protocols, and network security. Recent work has shown concrete benefits of DPU offloading for databases and KV stores [39, 60, 68]. However, these proposals only explored a narrow set of DPU capabilities, such as userspace networking, onboard DRAM, and power-efficient CPUs. They have been applied to only a few operation types, such as remote storage reads [68], indexing [60], and shuffling [39]. Another line of work benchmarks DPUs from *specific vendors* for *specific tasks*. For instance, DPUbench [44] measures the efficiency of MPI communication between DPUs, DPUBench [64] provides a benchmark suite for NVIDIA BlueField-2 DPU (BF-2), and Wei et al. [65] and Liu et al. [40] evaluated BF-2 for RDMA and computational efficiency, respectively. None of these studies targets database workloads. Moreover, each benchmark targets one device type, overlooking the heterogeneous hardware space, such as NVIDIA BlueField [6], AMD Pensando [2], Intel IPU [3], Marvell OCTEON [5], and AWS Nitro [8].

Therefore, there is a gap in existing benchmarking works that (1) targets cloud database systems and workloads and (2) produces findings that can be generalized to DPUs across vendors and generations to help understand the implications of DPU offloading for cloud databases. This is, however, a non-trivial task. DPUs exhibit high degree of hardware heterogeneity. Besides varying processor, network, and storage configurations, the availability of ASIC accelerators, such as those for deflate compression and RegEx matching, is highly hardware-dependent. Furthermore, the SDKs for accessing and programming the accelerators on DPUs are also vendor- or product-specific, such as Data-center-On-a-Chip Architecture (DOCA) for BlueField [50] and extensions to base Linux SDK for OCTEON [43]. This creates a hurdle for conducting standardized benchmarking across DPUs in order to obtain meaningfully comparable results.

To address these challenges, we developed a DPU-centric benchmarking framework, dpBento, which incorporates a *task abstraction* and a modular and extensible design. The task abstraction provides the necessary standardization across heterogeneous DPU hardware for the various performance tests targeting different aspects of data processing, which also serves as the ground for well-defined performance metrics and analyses. The modular and extensible design eases the implementation and integration of DPU-specific performance tests, enabling portability and benchmark automation. Building atop the dpBento framework, we developed benchmark tests for (1) primitive operations closely related to data processing efficiency, including those for compute, memory, storage, and networking, (2) cloud database system modules that verify

the benefits of DPUs, including predicate pushdown and index offloading, and (3) end-to-end database workloads on a popular lightweight DBMS (DuckDB).

We then use DPBENTO to conduct comprehensive evaluations of data processing capabilities on both NVIDIA (BlueField-2 and BlueField-3) and Marvell (OCTEON TX2) DPUs. For general workloads, such as primitive compute operations, we compare results across these DPUs and the host; we also evaluate the benefits of DPU-specific features for the corresponding tasks. Our evaluation reveals the following DPU offloading insights:

(1) The performance of general-purpose cores on DPUs has improved across generations. Although a state-of-the-art host CPU core remains more powerful, data width and data type matter for compute performance, and could lead to faster processing on DPU cores. This makes DPU cores attractive targets for offloading latency-sensitive operations.
(2) Throughput-sensitive operations can benefit from offloading to specialized hardware accelerators, and the performance depends highly on the data size distributions.
(3) DPU memory, albeit limited in capacity, can have performance on par with host memory, especially with a small working set and sequential operations. This makes DPU caching feasible to directly serve requests from the network.
(4) DPUs are efficient at storage and network I/O and thus offer opportunities to optimize I/O-intensive operations.
(5) Cloud database systems can opportunistically benefit from DPUs by, for example, offloading predicates for near data processing. However, naïvely executing full database queries on DPUs is consistently slow, motivating DPU-native designs to fully exploit the characteristics of the emerging hardware.

In summary, this paper makes the following contributions:

- We design a standardized and modular framework for benchmarking DPUs and a suite of tests for measuring data processing task performance centered on DPUs (§3) [1].
- We present results of benchmarking recent mass-production DPUs and provides insights into the compute, memory, and I/O efficiency (§4—§6) and the implications of in-network offloading of cloud database modules to DPUs (§7).

2 DATA PROCESSING UNITS

DPUs are programmable network interface cards (NICs) or SmartNICs that allow for data-path customizations and optimizations based on System-on-a-Chip (SoC) architectures. The built-in data-path operations include packet processing and disaggregated storage protocols such as NVMe-oF. They can be characterized by a set of hardware resources (see Figure 1). (1) As a NIC, a DPU provides state-of-the-art network speed, achieving 100s of Gbps bandwidth to remote host or DPUs. A DPU is also equipped with (2) an onboard low-power CPU (e.g., Arm) and a standard OS (e.g., Linux) for ease of access. To accelerate popular yet compute-intensive data-path tasks, such as compression, encryption, and RegEx matching, a DPU hardens these tasks as (3) accelerators to achieve superior performance and efficiency. (4) A DPU also has a moderate amount of main memory, typically ranging from 8 GB to 32 GB. (5) A PCIe switch on a DPU gives it access to external system-wide resources, such as host memory and peer PCIe devices (e.g., SSDs and GPUs). DPUs also have (6) auxiliary local storage (e.g., an SSD) for its firmware and OS, user programs, and necessary files.

[1] Artifacts are available at https://github.com/fardatalab/dpBento.

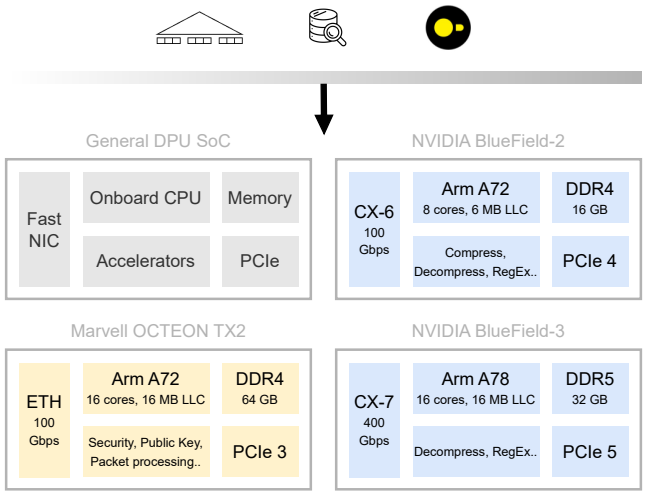


Figure 1: Overall architecture and variety of DPUs. It is practical and increasingly popular to offload data processing operations and system modules to the emerging hardware.

Compared to ASIC-based and FPGA-based SmartNICs, DPUs offer general programmability with onboard CPUs, and higher compute performance and energy efficiency with hardware accelerators. Given current trends in cloud infrastructure such as virtualization [25], high-performance I/O [32, 51], and disaggregation [26, 68], DPUs are becoming a popular solution to offload host functionalities to lower TCO and improve data-path efficiency [11, 48, 52, 66]. Recent work has shown promising results for offloading certain data processing tasks to DPUs [33, 60, 68].

3 METHODOLOGY

This section first describes the challenges of benchmarking heterogeneous DPUs from different vendors, then provides details on our DPU-centric benchmark suite and the workloads and hardware setup used for our experiments and analysis.

3.1 Challenges of Benchmarking DPUs

A crucial consideration of DPU offloading for data processing is the variety of DPUs, which is reflected in the following aspects.

- **C1: Resource Variety.** A DPU has resources for general and specialized computation, memory, storage, and interconnections to the network and to other resources on the host. These resources can be utilized by a data processing system but have different performance characteristics than host servers. Moreover, the configurations of these resources, such as accelerators and memory capacity, varies widely across different DPUs.
- **C2: Device Variety.** Many commercial DPUs are being produced: NVIDIA BlueField [6], Marvell OCTEON [5], Intel IPU [3], AMD Pensando [2], Broadcom Stingray [1], Kalray MPPA [4], AWS Nitro [8], Alibaba CIPU [7], and Microsoft Fungible [27], and most cloud providers are deploying them. Figure 1 shows the

specs of NVIDIA BlueField-3 and Marvell OCTEON TX2, two state-of-the-art DPUs we benchmark in this work. Although they follow the same high-level architecture, detailed hardware configurations, e.g., the network interface and hardware accelerators, greatly differ. (Details are in §3.4.) Moreover, DPU SDKs are often vendor-specific. For instance, NVIDIA provides the Datacenter-On-a-Chip Architecture (DOCA) SDK [50] to program BlueField DPUs, while Marvell offers more standard Linux toolchains and Data Plane Development Kit (DPDK) for OCTEON [43]. Even for the same vendor, DPUs across generations can be configured differently. For instance, from BF-2 to BF-3, not only are the CPU, memory, PCIe, and NIC upgraded, but the set of hardware accelerators has also been changed. Implementing tests to benchmark different DPU devices is a largely repetitive yet varied process.

3.2 DPU-centric Benchmarking

Benchmarking state-of-the-art DPUs is the key to understanding the behavior of various DPU resources and their potential capabilities for cloud data processing. To address the challenges of DPU variety, we develop dpBento, a DPU-centric benchmarking suite. A data processing workload in dpBento is implemented as a *task*, which encapsulates a well-defined unit of work in the database context, e.g., numerical arithmetic, memory pointer-chasing reads, or a database module, such as an index; it is also parameterized, both for the benchmark configuration such as whether to test random or sequential access, and for the metrics of interest such as throughput or latency. The task abstraction makes standardization of the benchmark possible, creating a unified configuration and result reporting for a concrete practical workload. By defining such tasks, benchmarking and analyses can stress and focus on specific aspects of the DPU hardware. This addresses the challenge of **resource variety (C1)**: dpBento offers an extensive set of tasks that cover various workloads and stresses the wide variety of hardware resources available on DPUs.

dpBento supports modular and composable stages for all performance tests, thus enabling automation of the benchmarking process under the framework illustrated in Figure 2. A benchmark job specifies tasks to be tested, along with their configuration parameters. For each task, it generates all permutations of parameters as individual runs of the tests. ❶ the `prepare` stage initializes the DPU environment as needed for all tests. Then, ❷ the parameter permutations are passed along with the metrics, to the `run` stage to conduct a test. Tests may generate intermediate logs of measurement results, which will be collected by dpBento. After all tests are executed, ❸ the `report` stage processes intermediate results and produces the final performance report. Finally, ❹ the `clean` stage is performed to remove all side effects such as configurations and datasets incurred during benchmarking. Benchmarking is automated by this workflow. To benchmark a specific DPU, one only needs to declare tasks—all the above steps are transparently managed by the framework.

This modularity facilitates DPU-specific hardware accelerator benchmarking, as well as ad-hoc data processing tasks. Database modules that a user considers offloading to the DPU, such as an index, can be implemented and then easily integrated into and executed by the benchmark framework automatically, in a modular

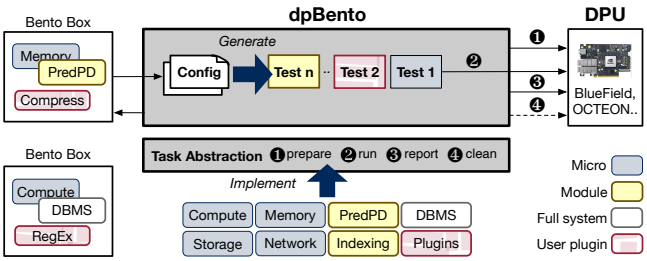


Figure 2: Enabling extensive data processing performance benchmarking on DPUs by addressing hardware variety.

and composable fashion. The preparation, reporting, and cleaning stages can be the same across DPUs. Only the running stage requires a separate implementation for a device-specific hardware accelerator. This addresses the challenge of **device variety (C2)**: for tasks that test generic hardware resources, implementations can be shared; for testing device-specific resources with vendor-specific SDKs, it is possible to only provide the minimal necessary implementation to support the automated benchmarking process.

dpBento is a key enabler of this benchmarking study. It is also extensible, allowing extension to the benchmark suite, potentially for new DPUs and hardware resources. By simply creating a dedicated directory under the framework, defining a set of test-specific parameters, and providing the necessary implementations that will be loaded as modules and function as benchmark drivers, any performance test can be incorporated into the framework for automated benchmarking and reporting.

3.3 Workloads

We have implemented a range of data processing workloads using the task abstraction of dpBento, as shown in Table 1. We categorize the performance tests into three main levels: **micro-tasks**, which focus on primitive operations related to computation, memory, and IO; **module-tasks**, which include high-level system components adopted in cloud DBMSs; and **a full-DBMS-task**, which tests end-to-end query execution. These tasks and the rationale for selecting them are presented below.

3.3.1 Compute. Compute micro-tasks include primitive operations covering numerical (integer and floating-point) arithmetic and string operations. These tasks are parameterized over data types, data widths or sizes, and operations performed, evaluated by throughput. We also implement a set of *optimizable tasks* that are higher-level compute-intensive operations (e.g., compression and RegEx matching). These are amenable to various CPU optimizations such as SIMD and multi-threading, and more importantly, to DPU hardware accelerators.

Rationale. All numerical and byte-string types are supported as attribute types by most databases. Wide integers such as `int128` are often used as the backing store of fixed-point decimal types (as an integer scaled by a power of ten [14]), while narrow ones like `int8` are commonly utilized for techniques like run-length or dictionary encoding. With the rise of vector databases and ML-for-DB, we anticipate floating point types will become more heavily used for similarity search [12, 34, 42] and inference. String operations such

Table 1: Benchmarking tasks implemented with DPBENTO.

	Tasks	Parameters
Micro	Compute	Data type, Operation
	Memory	Operation, Object size, Pattern, #Threads
	Storage	I/O type, Access, Pattern, Depth, #Threads
	Network	Message size, Depth, #Threads
Module	Pred pushdown	Scale, Selectivity, #Threads
	Index offloading	Scale, Op, Pattern, Split ratio, #Threads
Full system	DBMS	Scale, Query, Execution mode, #Threads

as comparisons and sub-string matching are common when evaluating predicates, and directly affect scan and filter performance. The selected set of optimizable tasks are frequently executed in cloud database systems [20].

3.3.2 Memory. Memory micro-tasks evaluate the speed of accessing in-memory objects by creating a memory buffer of a certain size, then performing pointer chasing based on specified access pattern (i.e., random or sequential), and measuring the throughput in accesses completed per second. In addition to these parameters, we vary the number of threads that perform memory accesses in parallel to explore the scaling of effective memory bandwidth.
Rationale. Database workloads typically involve many pointer-like accesses across different sizes of working sets: both KV-stores and relational databases encounter this during fundamental operations such as index/tree traversal, hash table lookups, and joins; each will have different memory footprints depending on the workload—the memory benchmark helps us understand how well DPUs fare in different scenarios.

3.3.3 Storage. These micro-tasks generate disk I/O activities parameterized to measure DPU-local storage device performance. At the core is an extensive storage testing toolkit that initialized files with random content on disk, issues reads and writes, and measures detailed latency and throughput statistics of these operations. To perform efficient storage accesses, it utilizes asynchronous I/O with `io_uring` or `libaio`. We test with four tuning parameters: I/O types (reads or writes), access granularity (in bytes), queue depth (number of outstanding IO requests), and concurrency (#threads issuing I/O in parallel).
Rationale. DPU-local disks have plenty of free capacity, ranging from tens to hundreds of GBs. The main use case is for users to store their programs and related files, but we want to investigate if the spare capacity is suitable for I/O-intensive workloads commonly executed by database systems.

3.3.4 Networking. To evaluate network communication performance, we include benchmarks to measure the speed of TCP and Remote Direct Memory Access (RDMA). For both cases, this benchmark task involves creating two communication endpoints, where they send and bounce messages of a certain size, measuring the end-to-end latency and throughput of transmissions. Parameterization includes the message size in bytes, queue depth (the number of outstanding messages), and concurrency (the number of TCP or RDMA connections, each processed by a separate thread).
Rationale. TCP sockets and RDMA are the most widely used communication stacks covering almost all existing data processing systems [13, 15, 59, 67]. Message sending and RPC semantics are also heavily used by cloud and disaggregated databases (e.g., sending large pages from storage servers or small control messages); throughput and latency are both important metrics of performance depending on the workload.

3.3.5 Cloud Database Modules. Prior work has demonstrated two general advantages of offloading cloud database components to DPUs [39, 60, 68]: near-data processing, which executes operations closer to data to minimize data movement overhead, and augmented processing in addition to the host. We include two tasks that offload cloud database modules to examine these benefits respectively.

Predicate Pushdown. This task targets a disaggregated storage architecture. SQL queries are executed by the database engine on the compute servers, while database files are stored on storage servers equipped with a DPU. Servers are interconnected by high-speed network. We execute and benchmark the scan-and-filter module on the DPU, returning only qualified tuples to the compute server. This task allows varying the scale of the input table stored on the storage server, the selectivity of the predicate, and how many concurrent workers are spawned on the DPU. The metric is tuples scanned per second.

Index Offloading. The second module we offload to the DPU is the index. The DPU on a database server acts as a coprocessor of the host to partially serve index lookups. A B+ tree is range-partitioned between the host and the DPU such that serving requests from the DPU can augment the overall lookup throughput. The implementation is adapted from LMDB [58], which supports concurrency with MVCC. This task uses the YCSB benchmark as the workload, and allows varying the index size (KV record size and count), operation (read/write), access pattern (uniform or skewed), the ratio of the index offloaded, as well as the number of DPU cores participating in co-processing, measuring the throughput in lookups per second.

3.3.6 A Full DBMS. Some recent work advocated replacing host servers entirely with DPUs to manage hardware resources [30, 53]. This architecture makes DPUs standalone execution environments, rather than co-processors of hosts. To investigate the implications, we include a system evaluation task with DuckDB [23], a full-fledged lightweight RDBMS. Specifically, this benchmark task runs DuckDB on the TPC-H dataset, parameterized over queries to run, the number of threads utilized on the DPU for DuckDB, and the execution mode (cold, where queries are first-time executed on the DPU, or hot, where queries have been executed a few times to warm up internal structures). We use query execution time as the performance metric.

3.4 Benchmarked Hardware

In this study, we benchmark several recent DPUs that are mass produced. This section describes the hardware setups of the DPUs, as well as the host machine that we use as the baseline against which to compare the DPU's data processing performance.

NVIDIA BlueField. We assess two generations of NVIDIA BlueField DPUs: BlueField-2 (BF-2) and its successor BlueField-3 (BF-3).

Both are in mass production [40, 65, 68]. Figure 1 shows the details of the two DPUs in our testbed. On BF-2, there is an Arm A72 CPU (64-bit), which consists of 8 cores @ 2.5 GHz. Every two cores share a 1 MB L2 cache, and a 6 MB L3 cache is shared between all cores. There is 16 GB onboard DDR 4 memory that serves as the DPU's main memory. It has a variety of hardware accelerators, including those for compression, decompression, and RegEx matching. A ConnectX-6 NIC provides 100 Gbps network bandwidth, and the board is connected to the host via PCIe 4.0. BF-2 also has an eMMC flash device that is soldered directly onto its motherboard.

BF-3 is an upgrade from BF-2 in nearly every aspect: the Arm CPU is upgraded to A78, which has 16 cores with clock rates up to 3.0 GHz. L2 cache increases to 6 MB and L3 to 16 MB. The onboard memory levels up to DDR 5, 32 GB. The network interface is upgraded to ConnectX-7, providing 400 Gbps bandwidth, and the interface to the host is advanced to PCIe 5.0. The directly attached local storage is also updated to a 160 GB NVMe SSD. Interestingly, the compression engine is removed, which makes a different set of hardware accelerators.

Marvell OCTEON. Also shown in Figure 1 is a Marvell OCTEON TX2 DPU in our testbed. Same as BF-2, it is equipped with an Arm A72 CPU but with more cores (24 cores @ 2.2 GHz), with a 1 MB L2 cache shared between two cores and another 14 MB L3 cache shared across all cores. The DPU has 32 GB onboard DDR 4 memory, and provides a 100 Gbps Ethernet network interface and PCIe 3.0 for connecting to the host. The hardware accelerators are different from BlueField, primarily for network security and packet processing. Like BF-2, our OCTEON has an eMMC local storage of 64 GB.

Host Machine. We conduct the same set of DPBENTO tasks on a host server, which was recently assembled, and thus represents a state-of-the-art host configuration. Specifically, the server has two AMD EPYC 9254 24-Core CPUs @ 2.9 GHz, so in total there are 48 cores (96 hyperthreads), 48 MB L2 cache, and 256 MB L3 cache. The main memory on the server is 128 GB DDR 5, and for persistent storage, there are two 960 GB NVMe SSDs. Comparing the benchmark results to those on the host helps better interpret data processing performance on the DPUs.

4 COMPUTE EFFICIENCY

We first present the benchmark results of the compute tasks on the three DPUs and the host. Detailed findings follow.

4.1 How weak are general-purpose DPU cores at primitive computing operations?

Highlighted Findings

- *DPUs are faster at processing smaller operands and can even outperform the host for floating-point processing.*
- *For strings, DPUs are more suitable for simpler operations.*

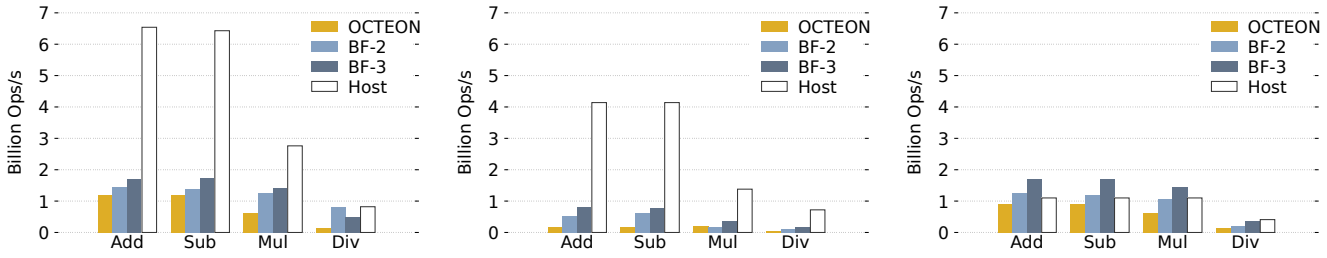
Benchmark Setup. Table 2 shows the testing parameters that we use to benchmark DPUs with the primitive compute tasks: arithmetics on integers and floating-points of different widths, and byte string operations of different sizes. These are commonly seen in database systems, such as expression evaluation and predicate filtering. It is natural to assume that with a low powered SoC, the performance of compute-bound operations are much worse on the DPU than on the host, but exactly how wide is the gap?

Table 2: Experiment parameters used for benchmarking the compute performance of DPUs with primitive operations.

	Data Type	Operation
Arithmetic	`int8`, `fp64`, `int128`	`add`, `sub`, `mul`, `div`
String	`str10`, `str64`, `str256`, `str1024`	`cmp`, `cat`, `xfrm`

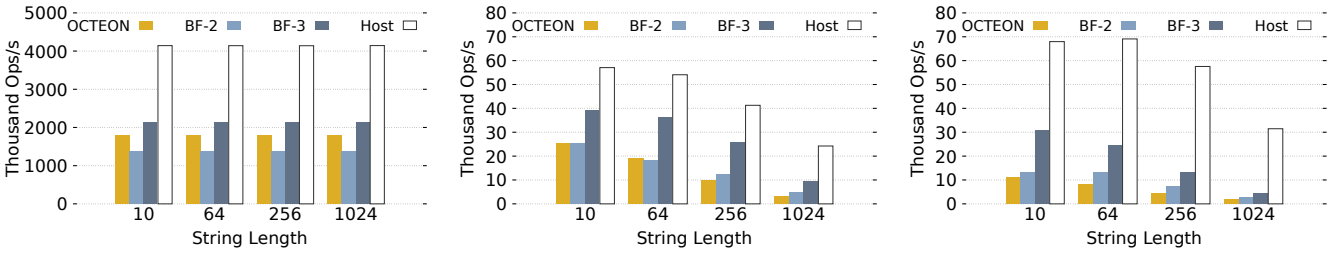
Detailed Results. The detailed benchmark results are shown in Figure 3 for both integers and floating-point numbers. First, Figure 3a shows the performance of the 8-bit integer operations. For `add` and `sub`, the host AMD CPU demonstrates superior performance advantages compared to the DPU Arm cores, achieving 6.5 billion operations per second (ops/s)—up to 5.5× higher than the DPUs. For more complex `mul` and `div`, all four CPUs exhibit a lesser throughput as expected, but the degree of which is different for the host CPU compared to the DPUs' CPUs. The host CPU experiences a 58% throughput decrease when performing `mul` compared to `add` on `int8`, higher than the degradation seen on OCTEON (49%) and much higher than that of BF-2 (14%) and BF-3 (19%). However, the host CPU is still 2× better than the best DPU (BF-3). For `div`, a further decrease in performance can be observed on the host CPU (70% throughput drop compared to `mul`). BF-2 and BF-3 suffer relatively less of a throughput degradation from `mul` (36% and 64%, respectively), while OCTEON suffers more at 80%. With wider operands (`int128`), as shown in Figure 3b, we observe similar trends in performance across the board—the host is still the best performer by a large margin, and all CPUs see their throughput drop in `mul` and `div` compared to `add` and `sub`, with DPUs seeing larger percentage drops across the board. The effect of increased operand width is more pronounced for DPUs, resulting in a decrease of 63% to 76% compared to 34% on the host. The host–DPU gap widens further with `mul` and `div`: the drop on DPUs are 63%–77%, while only 12% on the host, which is now 4.7× faster than the best-performing DPU, compared to 1.7× on `int8`. These results show that the DPUs are better at handling smaller operands. Floating point (`fp64`) performance paints a very different picture for the DPUs, as seen in Figure 3c. The highly noticeable performance lead of the host CPU in integer compute is almost completely lost, where the two generations of BlueField DPUs now outperform the host CPU in `add`, `sub`, and `mul`, where BF-3 leads by more than 50% on average. OCTEON also becomes much more competitive with floating point compute compared to its integer performance, albeit still trailing behind the BlueField family of DPUs and the host CPU by a fair margin. The host CPU still holds an advantage with FP division, although the gap is much smaller than that of integers. The high performance of DPUs on floating-point operations can be ascribed to the hardware support in the Arm architecture [10].

Besides numerical operands, we also benchmark three typical byte-string operations: comparison (`strcmp`), simple manipulation (`strcat`), and complex transformation (`strcfrm`), and Figure 4



(a) Results on `int8`. (b) Results on `int128`. (c) Results on `fp64`.

Figure 3: Primitive arithmetic operations on integers and floating-point numbers.



(a) Results on `strcmp`. (b) Results on `strcat`. (c) Results on `strxfrm`.

Figure 4: Primitive string operations.

shows the results. The host CPU has higher throughput in all categories, but the performance gap between the host CPU and DPUs varies across categories. For string comparison, string size matters little for DPUs and the host CPU, where the host CPU has close to 2× the performance of the most powerful DPU (BF-3). For string manipulation, the host still has the absolute performance advantage, but the lead over DPUs are smaller, especially for small string sizes: BF-3 achieves 68% the performance of the host CPU, with 10 B strings, dropping to 39% for 1024 B strings. String transformation operations paint a different picture—the gap widens in favor of the host CPU as the string size increases, and that BF-3 holds a more substantial lead over the other DPUs than that of string search. However, the host CPU still has more than two times the performance of BF-3, where the gap widens with larger string sizes, reaching more than 7× the throughput on OCTEON.

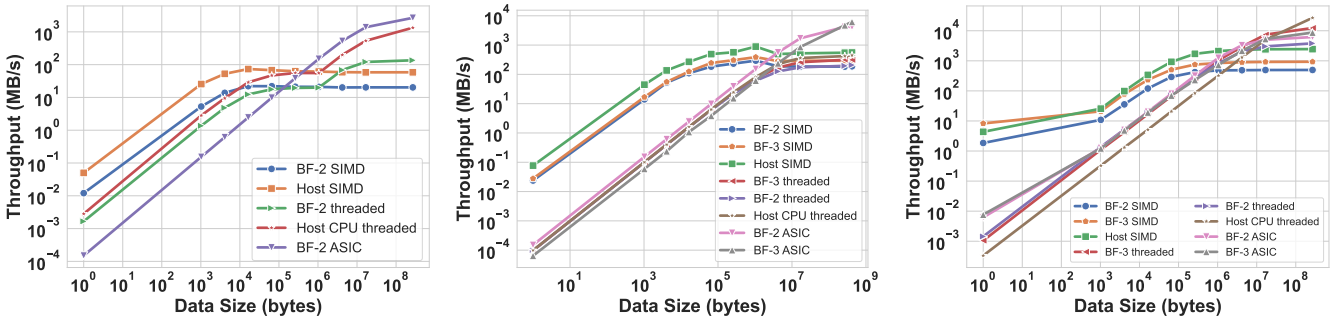
4.2 How powerful are specialized accelerators compared to general-purpose cores?

Highlighted Findings

- *Hardware accelerators do not always outperform CPUs. Even when they do, they can improve throughput, not latency.*
- *Offloading compute-intensive tasks to hardware accelerators can save many CPU cycles.*

Data Compression. We first examine compression using DEFLATE [22], one of the most commonly used compression algorithms in database systems [31]. Specifically, we use this algorithm to compress strings generated from TPC-H `orders` table of specified size. BF-2 offers a hardware accelerator for this compression. In addition to single-core performance of BF-2 and the host, we also compare BF-2 hardware acceleration to multi-threading (all cores) and SIMD, two optimizations that boost the performance on CPUs. Figure 5a shows the compression speeds of different hardware techniques. We first observe that the DPU hardware acceleration is not always desirable: there is a fixed startup overhead when invoking the hardware accelerator on BF-2. For data below 100 KB, the performance of hardware offloading is lower than that on host and BF-2 CPUs. However, when data size increases (≥1 MB), BF-2 hardware offloading shows superior performance, offering higher throughput than threaded execution on the host CPU (4.9× faster when compressing 512 MB data). We note that for very small data sizes, multi-threaded execution also provides no benefits due to threading overhead.

Data Decompression. Decompression acceleration is supported by both BF-2 and BF-3, and the benchmark results are shown in Figure 5b. Similar to compression results, the hardware accelerators on the BlueFields incur a relatively high startup latency, which is reflected in low throughput for smaller payload sizes (lower than 1 MB), but are much more efficient with larger sizes: BF-2 decompression accelerator is 13× / 21× faster than the host CPU / its onboard CPU with multi-threading when compressing 256 MB data. Additionally, the BF-3's accelerator exhibits a higher startup latency than that on BF-2, but becomes faster than its last-generation counterpart as the data size grows to 100s of MB. Another observation is that for decompression, the performance gap between the host



(a) DEFLATE compression. (b) Decompression. (c) RegEx matching.

Figure 5: Optimizable tasks that can be accelerated by hardware techniques.

and onboard CPUs is relatively smaller, especially with threaded execution, this is likely due to the nature of DEFLATE algorithm, where decoding serializes data access and is thus hard to parallelize.

RegEx Matching. Our final optimizable task is RegEx matching, where both BF-2 and BF-3 offer hardware acceleration. In this task, we match the pattern that is part of TPC-H query 13, specifically, the generated pattern is `like '%special%requests%'`.

Figure 5c shows the result of varying the size of the string dataset. First, the hardware-accelerated performance of BF-2 and BF-3 is identical, which is better than using threaded execution for small data sizes. However, a single-threaded implementation with SIMD provides much better performance. As data size increases, the hardware accelerators achieve comparable performance to multi-threaded execution (with all available CPU cores). However, eventually the latter scales better: on 256 MB data, the host CPU and BF-3 CPU are 3× and 1.4× faster than the RegEx accelerator on the two DPUs, respectively. Using all cores for RegEx matching may not always be feasible or desirable—in that case, DPU hardware accelerators handily lead over the single-threaded SIMD counterpart and provides considerable CPU savings.

5 MEMORY OPERATIONS

Highlighted Findings

- *DPUs are good at sequentially accessing in-memory objects. Sometimes, they even outperform the host.*
- *If the memory accesses are random, DPUs are better at accessing small objects than the larger ones.*
- *DPUs' limited core count can become a bottleneck for high-throughput memory access.*

Bemchmark Setup. The configuration used to benchmark memory performance of DPUs with the memory task is listed in Table 3. We use one core to issue random and sequential reads and writes to different memory sizes as shown in Figure 6, and additionally investigate their scaling behavior as shown in Figure 7.

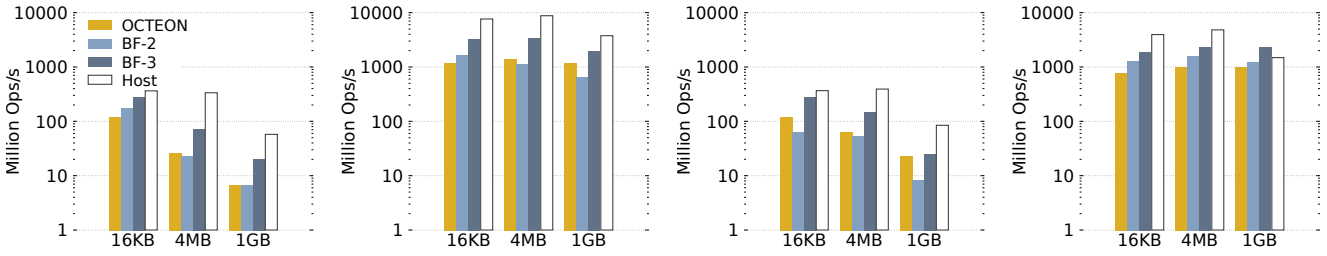
Detailed Results. Random access patterns occur frequently in database workloads, such as tree traversal, table lookups, hash

Table 3: Parameters for memory benchmarking.

Operation	Object Size	Pattern	#Threads
Read, Write	16 KB, 4 MB, 1 GB	Random, Sequential	1–Max

table insertions, etc. The prevalence of random accesses means that the performance of many basic operations depend on the DPUs' ability to sustain a high throughput, which would affect the overall performance of potential DPU offloading of such workloads to a large extent. To this end, we evaluate random access throughput for both reads and writes, across small, medium and large working set sizes. Figure 6a and Figure 6c shows the throughput of reads and writes respectively for random accesses. When performing random reads to an object as small as 16 KB, which fits into the L2 cache on all platforms, thus the accesses are efficient: all can achieve higher than 100 million ops/s random access throughput. Between DPUs, BF-3 achieves the highest throughput, 1.6× higher than BF-2, this gap is even larger than that between the host and BF-3 (1.3×). When increasing the object size to 4 MB, the throughputs of the DPUs drop dramatically: 78%, 87%, and 75% decrease for OCTEON, BF-2, and BF-3, respectively. At this size, the working set would spill to to L3 on the DPUs, but as the host CPU has a much larger L2 cache (48 MB), its throughput remains high. To eliminate CPU caching effect, we further test with 1 GB, which exceeds the CPU caches to a large extent. We can see that the throughput drops to a lower level for all platforms: the throughput of the host drops to 58 million ops/s—a 83% drop, followed by BF-3, achieving 20 million op/s, and then by OCTEON and BF-2, both at 6.7 million ops/s. This experiment shows that for random reads, DPUs are good at caching small objects with comparable performance to the host. The above findings apply to random writes as well, with small differences in absolute numbers, as all platforms witness throughput drop when accessing objects larger than their L3 cache. The best-performing DPU remains BF-3, whose gap to the host enlarges as the memory buffer size increases. Interestingly, OCTEON now performs significantly better than BF-2 and approaches BF-3 for writing to a 1 GB buffer.

Sequential access is also crucial to many scenarios, such as data structures optimized for such cases, like MemTables of LSM-trees,



(a) Random reads. (b) Sequential reads. (c) Random writes. (d) Sequential writes.

Figure 6: Memory efficiency of DPUs with varying object sizes.

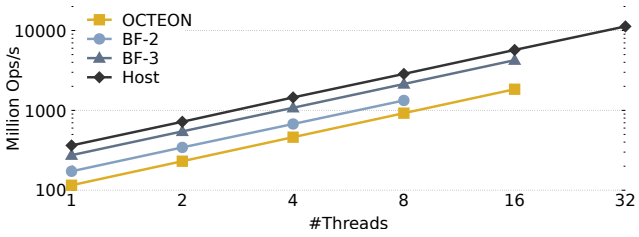


Figure 7: Scaling up memory accesses (random reads). BF-2, BF-3, and OCTEON have 8, 16, and 24 cores, respectively.

and is generally applicable to processing on the data plane (such as in-network aggregation or filtering), which DPUs are expected to excel at. Similarly, we evaluate reads and writes throughput across different working set sizes for sequential accesses as well. Figures 6b and 6d show the performance of reads and writes, which are much higher than random accesses, as expected since hardware prefetching plays an important role. We make the following observations. Firstly, prefetching on the DPUs is as effective as on the host: the throughputs of all platforms remain largely on the same level when varying the object sizes from 16 KB to 1 GB for both reads and writes. Secondly, the gap between the DPUs and the host is not as large as that when performing random accesses, especially for medium and large sizes, e.g., the host is now 5.9× faster than BF-2 for sequential reads (vs. 8.6× for random reads). Finally, DPUs can even achieve higher throughput than that of the host, highlighted by the 1 GB sequential writes case of BF-3, which achieves 2.2 billion ops/s compared to 1.5 billion ops/s on the host. In particular, when sequentially writing to a 1 GB memory, BF-3 achieves 2.2 billion ops/s, which is higher than the host throughput at 1.5 billion ops/s.

Tests above are conducted for a single core, showing current-gen DPUs can match and even exceed the performance of the powerful host system. However, how well can the memory subsystem scale up with concurrent accesses and keep up the performance? Figure 7 demonstrates how CPU core count affects memory throughput, where multiple cores randomly access a 16 KB memory buffer in parallel. We observe that a single thread is incapable of saturating memory bandwidth, and the achieved throughput increases linearly with core count. However, the DPUs have fewer cores (8, 16, and 24 on BF-2, BF-3, and OCTEON respectively, vs. 96 on the host), which limits their peak memory throughput: 1.3 billion op/s on BF-2, 2.7 billion op/s on OCTEON, and 4.3 billion op/s on BF-3; in comparison, the host achieves 11.3 billion op/s with 32 cores.

Table 4: Parameters for storage benchmarking.

I/O Type	Access Size	Pattern	Queue Depth	#Threads
Read, Write	8 KB–4 MB	Random, Sequential	1–256	1–Max

6 I/O PERFORMANCE

Next, we evaluate the I/O performance, including both storage and networking on the three DPUs, and compare it to that of the host.

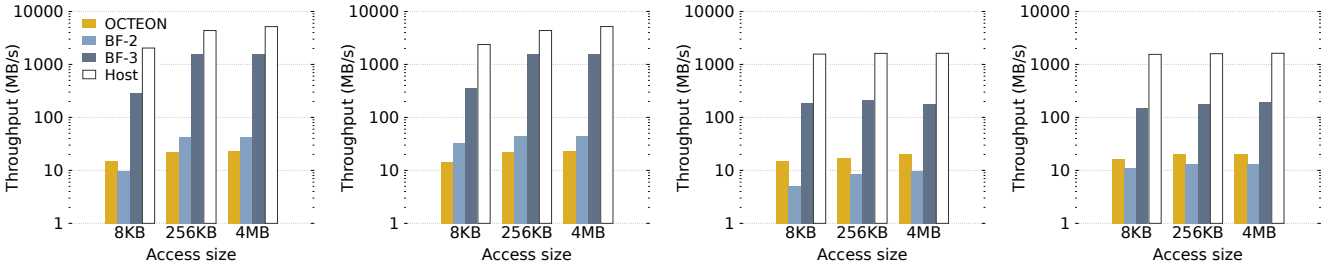
6.1 How can the on-device storage be useful?

Highlighted Findings

- *DPUs generally provide considerably lower storage performance than the host for throughput-bound and large I/Os.*
- *The latest DPU achieves low latency for fine-grained accesses.*

Benchmark Setup. The testing parameters are shown in Table 4. Like memory benchmarking, we evaluate the performance of both sequential and random reads and writes. We vary access granularity, the number of outstanding requests, and parallelism. Since storage I/O is asynchronous, we measure both latency and throughput.

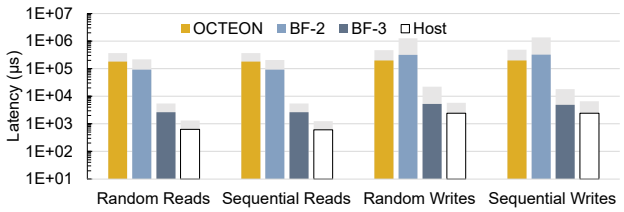
Detailed Results. We first tune the above parameters on each DPU and the host to achieve its highest storage I/O throughput. The results are shown in Figure 8. Across all settings, we observe three levels of performance: the slowest eMMC flash devices on OCTEON and BF-2 (10s–100s MB/s), the faster NVMe SSD on BF-3 (100s–1000s MB/s), and the best-performing NVMe SSD on the host (1000s MB/s). While the NVMe SSD on BF-3 is much faster than the storage on other DPUs, the gap to the host storage device remains large—2.8×–10.5× slower. The host storage performance is two orders of magnitude higher than that on the slower DPUs. The impact of storage testing parameters varies across platforms. When performing random reads, increasing the access size from 8 KB to 4 MB reduces the degree of randomness (since bytes within



(a) Random reads. (b) Sequential reads. (c) Random writes. (d) Sequential writes.

Figure 8: DPU and host local storage throughput. We vary the I/O access size from 8 KB to 4 MB, and evaluate the storage I/O throughput under random/sequential read and write patterns.

(a) Latency of 8 KB accesses.



(b) Latency of 4 MB accesses.

Figure 9: Storage I/O latency. Foreground bars of different colors represent average latency, and background light grey bars represent p99 tail latency.

an access are sequentially read from the device), which is more beneficial for BF-2 and BF-3 than for OCTEON and the host (350% and 440% vs. 50% and 150% throughput increase, respectively). When changing the access pattern completely from random reads to sequential reads, the throughput of BF-2 increases significantly for small accesses: a 250% increase for 8 KB reads. The benefit is not reflected in other platforms: only a 17% difference is observed on the host, which shows the efficiency of SSDs in random accesses. Writes are generally slower than reads (Figures 8c and 8d vs. Figures 8a and 8b). Similarly to reads, OCTEON and BF-2 are slower than BF-3 and the host in writes, and different access sizes and patterns impose the highest impact on BF-2. The gap between BF-3 and the host is larger than in reads.

Figure 9 shows the storage I/O latency. In this experiment, we set the number of outstanding requests and the number of threads both as one to achieve the lowest latency on each platform. Figures 9a and 9b report the average and tail latencies of the 8 KB and 4 MB accesses, respectively. We make more promising observations than

Table 5: Parameters for network benchmarking.

Message size	Queue Depth	#Threads
32 B – 32 KB	1–128	1–Max

in the throughput results: the latency of DPUs (particularly BF-3) can be on par with and even lower than that of the host. For small random and sequential reads (Figure 9a), the tail latency of BF-3 is ~20% lower compared to the host. Its average latency is also lower in serving random reads. This is especially advantageous for serving remote storage requests considering the shorter distance to the network interface on the DPU from the DPU storage device. However, when accessing larger blocks (4 MB), where performance becomes bandwidth-bound, Figure 9b shows that even on BF-3, the time is 3×–5× higher than on the host.

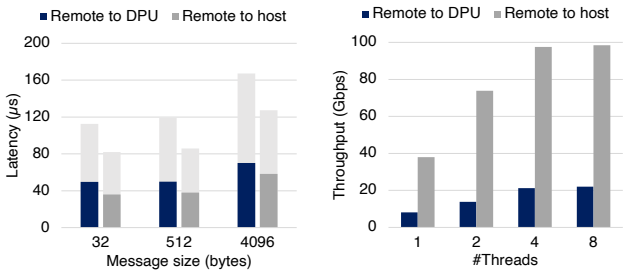
6.2 How efficiently can DPUs utilize the high-speed NICs for networking?

Highlighted Findings

- *Offloading network communication to DPUs with the on-board TCP stack reduces performance, especially throughput.*
- *Kernel-bypass networking can eliminate the impact of weak cores and achieve even lower latency than the host.*

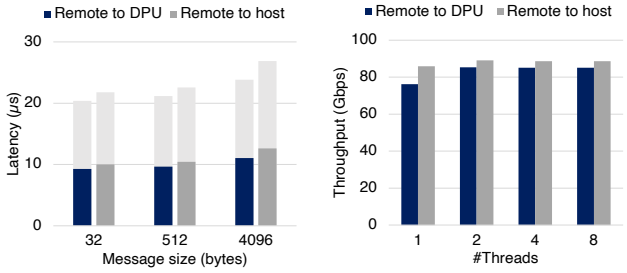
Benchmark Setup. In this test, we set up a physical network in which a remote server (with the same configuration as the host machine) connects to one of our DPUs (BF-2) via a 100 Gbps cable. The testing parameters are specified in Table 5.

Detailed Results. We first investigate the network latency. To do so, we spawn one thread on the remote server, which issues closed-loop ping-pong messages to accurately measure the network round-trip latency. We compare the latency between the remote server and the DPU to that between the remote server and the local host. This comparison seeks to assess the efficiency of the DPU for communication offloading. Figure 10a shows the average and tail latencies of different message sizes. We observe that generally the latency between the remote server ("remote" for short) and the DPU



(a) Latency (average and p99). **(b) Throughput.**

Figure 10: Network performance with TCP sockets.



(a) Latency (average and p99). **(b) Throughput.**

Figure 11: Network performance with RDMA.

is higher than that between remote and the host on all message sizes. On average, the former is 30% higher than the latter. We ascribe this latency overhead on the DPU to its wimpy CPU, where the Linux TCP/IP stack runs—software simply becomes slower.

We next measure network throughput, where the above finding is also reflected. For this measurement, we let the remote server and the DPU/host create multiple threads, each processing a connection for exchanging large messages (32 KB). Each connection maintains a queue depth of 128 to ensure that the connection throughput is saturated. Figure 10b reports the network throughput between remote and the DPU and between remote and the host. The trend is clear—the performance gap to the host is larger than in latency tests. With one thread, the DPU can achieve 8 Gbps throughput, while the single-thread throughput of the host is 4.8× higher at 38 Gbps. Both the DPU and the host saturate reach the peak throughput with four threads, where the DPU's throughput is 22 Gbps and the host's throughput is 98 Gbps. In fact, the single-thread throughput of the host is 1.7× higher than the eight-core throughput of the DPU. This result shows that offloading high-throughput network communication to the DPU can cause great performance degradation *using the TCP/IP stack in the onboard Linux*.

To verify that the above inefficiencies originate from the software stack running on the weaker cores, we implemented an additional plugin task in DPBENTO for network benchmarking with Remote Direct Memory Access (RDMA) over InfiniBand, supported by the BF-2 DPU. RDMA bypasses the onboard Linux to directly

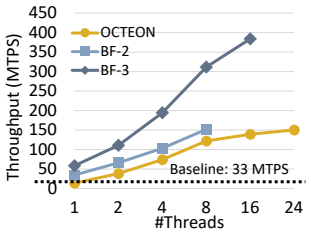


Figure 12: Filter pushdown.

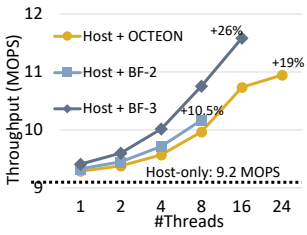


Figure 13: Index offloading.

access the NIC for data transfers. Specifically, the task uses NVIDIA-provided `ib_read_lat` and `ib_read_bw` tools to perform RDMA reads from the remote server to the DPU/host and measure network performance. We use the same parameters provided in the previous latency and throughput measurements. Figures 11a and 11b compare the kernel-bypass latency and throughput between remote and the DPU to that between remote and the host. We observe that when the networking stack in the onboard OS is bypassed, *network communication to the DPU has lower latency than to the host.* As Figures 11a shows, when accessing 4 KB data, the latency of the DPU is 12.6% lower than that of the host. The lower latency can be explained by the shorter distance from the NIC to the DPU memory than to the host memory. Regarding throughput, although the single-connection performance of the DPU is still lower than the host, the gap is marginal (11.3%). The peak throughput is achieved with two threads/connections for both the DPU and the host, where the throughput gap is further closed.

7 OFFLOADING CLOUD DATABASES

We now investigate the performance of running cloud DBMS modules and full DBMS queries on DPUs. These tasks involve a complex mix of the DPU resources, such as primitive compute, memory and networking, and more accurately reflect the capabilities of DPUs under real-world use cases.

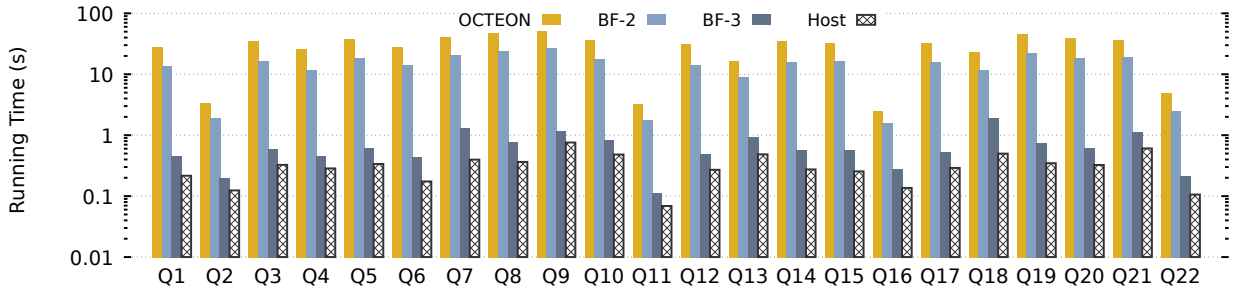
7.1 How well can DPUs support near data processing?

Highlighted Findings

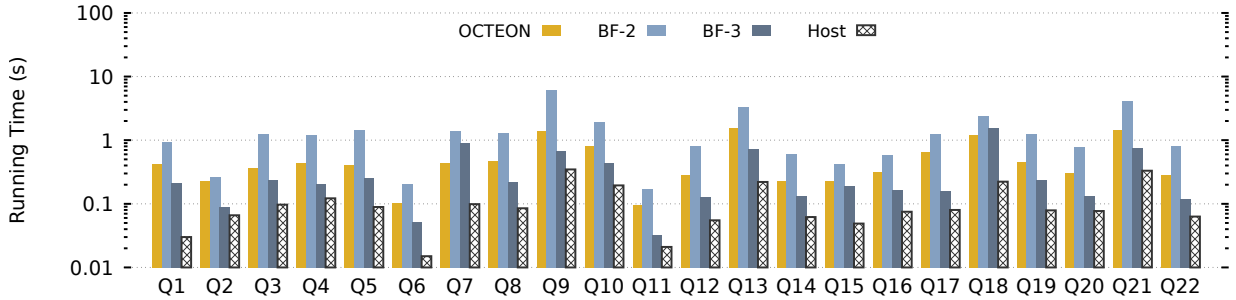
- *Even with the wimpiest DPU, near data processing can significantly speed up data scans on disaggregated storage.*
- *The latest DPU levels up the performance benefits to an order of magnitude.*

Benchmark Setup. We fix the scale of the database as 10 GB (TPC-H scale factor 10) and the predicate selectivity as 1%, and vary the number of cores utilized on the DPU for the scan from 1 to all available cores.

Detailed Results. Figure 12 shows the performance of the basic scan and DPU-enabled predicate pushdown. The baseline where the entire table is fetched from the storage server to the compute server incurs expensive network and storage I/O and thus has low scan throughput—33 million tuples per second (MTPS). Pushing down



(a) Cold execution.



(b) Hot execution.

Figure 14: Running times of DuckDB on different platforms.

the predicate to the DPU on the storage server can eliminate most of the data movement: the weaker DPUs, i.e., BF-2 and OCTEON, outperform the baseline when two cores are utilized for the scan, and when all their DPU cores are utilized, their throughput reaches 150 MTPS, 4.5× faster than the baseline. BF-3 is significantly faster than other solutions. Its speedup over the baseline is 1.8× with a single core and 12× with all its 16 cores. This result confirms the potential of DPUs for near-storage data processing.

7.2 How effective are DPUs as hosts' coprocessors?

Highlighted Findings

- *DPUs with higher core counts are more effective coprocessors to offload host workloads.*
- *Simply using the host's counterpart resources on DPUs leads to only marginal performance gains.*

Benchmark Setup. To evaluate the performance increase when using a DPU as a coprocessor of the host, we set up an index of 50 GB (50 millions of 1 KB records) and split it between the host and the DPU with a ratio of 10:1. We then execute random reads with a uniform distribution on the host and the DPU separately and measure the overall index throughput.

Detailed Results. Recent work has reported the performance increase by sharing index workloads between the host and the BF-1 and BF-2 DPUs [60], our experiments further examine this benefit on a more recent DPU (BF-3) and a DPU from a different brand (OCTEON). The results are shown in Figure 13. Without any offloading, the host can achieve 9.2 million operations per second (MOPS) with 96 threads. By offloading part of the index to the DPU, more requests can be serviced per time unit. Although each individual DPU core is weaker than the host CPU, fully utilizing the DPU leads to noticeable performance increase: 19%, 10.5%, and 26% higher throughput when offloading to OCTEON, BF-2, and BF-3, respectively. This result shows the benefits of having a higher core count on the DPU as a coprocessor: although OCTEON has weaker cores than BF-3, its final throughput increase is on par with that of BF-3. The significance of the additional throughput depends on the use case, but in general offloading host workloads with the counterpart resources (e.g., CPU and memory) on DPUs without exploring a DPU-native design (e.g., utilizing the hardware accelerators) contributes fractionally to the overall system performance.

7.3 How about completely handing off an existing DBMS to DPUs?

Highlighted Findings

- *Executing complex queries on DPUs is consistently slower than on the host.*
- *Due to high memory performance on DPUs, executing hot queries is more preferable.*

Benchmark Setup. We finally benchmark the DPUs using the macro-task with DuckDB. On each DPU platform, we allocate all available cores to DuckDB and measure its running time for each TPC-H query (scale factor 10) under cold and hot executions. Note that explicitly not our goal is advocating full system offloading on DPUs. Our results confirm the overhead of doing so and thus motivate co-designs between data systems and DPUs to incorporate partial offloading [39, 68].

Detailed Results. Figure 14a shows the results of cold executions. The host outperforms all DPUs, as expected. Its average query execution performance is 87×, 43×, and 2.1× higher than OCTEON, BF-2, BF-3, respectively. The primary bottleneck in this execution mode is disk I/O, particularly sequential reads as the tables are scanned and loaded into the main memory. Recall from Section 6.1 that the eMMC flashes on OCTEON and BF-2 are much slower than the NVMe SSDs on BF-3 and the host, which is reflected in the end-to-end query execution. Between the DPUs, BF-3 is 21× faster than its previous generation, and as Figure 8b shows, BF-2 achieves higher sequential read performance, and thus the average query processing time on BF-2 is 2× shorter than on OCTEON.

We make different observations with hot executions, which are illustrated in Figure 14b. Since disk I/O is avoided, the performance of CPU and memory now dominates query execution. As the gaps between the host and the DPUs on these resources are smaller, on average, the performance degradation is less significant than that in cold executions. As we observed in the previous experiment, DBMS query execution can benefit from higher parallelism enabled by more cores. This also explains the change between OCTEON (24 cores) and BF-2 (8 cores). The former, which was slower in cold executions, is now 2.7× faster.

8 RELATED WORK

DPU Benchmarking. DPUBench [64] proposes a benchmark suite that targets DPUs. The benchmark suite includes operators for storage, networking, and security. It also includes end-to-end applications that use the operators for communications, file compression and integrity checking, and security. However, DPUBench only evaluates BF-2. Even though DPUBench shares some operator benchmarks with dpBento, it is insufficient to evaluate data processing systems, which is the major contribution of dpBento.

DPU-Bench [44] evaluates DPU performance under HPC scenarios. Specifically, the benchmark suite measures RDMA-based MPI communication to BF-2. The goal is to determine the number of DPU processes to maximize offloading efficiency. The follow-on work [45] compares BF-2 and BF-3. dpBento, in contrary, focuses on DPU's capability to offload data processing tasks.

Wei et al. [65] quantitatively analyzed the performance of various RDMA paths on BF-2. Based on the benchmark studies, the work proposes an optimization guide, and applies the guide to prior DPU-accelerated file system and key-value store. Chen et al. [19] performed a more comprehensive performance studies, including both compute and communication, of BF-3, with a focus on the RISC-V data path accelerators (DPA). Unlike dpBento, both works study only a single DPU platform; their benchmarks are also inadequate to directly evaluate data processing applications.

Other prior works benchmark specific aspects of DPUs. Li et al. [38] evaluate the performance of lossy (SZ3) and lossless (DEFLATE, lz4, zlib) compression in the SoC and ASIC ("C-engine") implementations in BF-2 and BF-3. Liu et al. [40] use the `stress-ng` tests to evaluate BF-2. Zhou et al. [71] measure the performance of offloading microservices to an Intel IPU. Compared to these works, dpBento provides a more comprehensive benchmark suite for heterogeneous DPU platforms. dpBento also targets offloading for data processing systems, which presents a gap in the literature.

DPU Offloading. DPU has been a popular hardware target for application and system offloading. LineFS [35] offloads operations in a distributed file system to BlueField-2. Xenic [55] partially offloads data store and concurrency control to accelerate a distributed transactional system. Several prior systems [29, 47, 56] demonstrate the performance benefit of offloading network functions to DPUs. A line of work [41, 54] designs general programming frameworks to offload applications to DPUs. dpBento, in comparison, targets offloading data processing system components to DPUs.

9 CONCLUSION AND FUTURE WORK

We conducted comprehensive performance benchmarking on recent, mass-production DPUs with a range of data processing workloads from primitive compute, memory, disk, and network operations to higher-level cloud DBMS modules and full queries, enabled by a benchmarking framework. Based on the detailed results and analysis, we provided insights into the performance characteristics of DPUs for offloading data processing tasks, which are useful to guide further investigations of DPU offloading for cloud databases. In particular, we identify two promising future directions as follows.

High-performance Caching on DPUs. Our results reveal that DPUs offer excellent memory performance, especially with sequential access patterns and small working sets. Given the unique advantages of DPUs for processing network requests (every network packet flows through and can be processed by a DPU), caching small and critical distributed database components in DPU memory can enable new system optimization opportunities. A specific example is to cache lock table entries on DPU to reduce the latency of lock and release requests and further optimize locking protocols for distributed transaction execution.

DPU-native Cloud DBMS Design. This study confirms the overhead of fully offloading database workloads to DPUs. To unlock the potential of DPUs for cloud databases, a promising approach we believe is "DPU-native" design that best exploits the advantages of DPUs and avoids their inefficiencies. Examples of this design include partial offloading and hardware acceleration. For partial offloading, database query execution should be decomposed into more fine-grained tasks. Only tasks where DPU resources are sufficient and efficient, e.g., I/O and near data processing, should be offloaded. For hardware acceleration, the specialized accelerators on DPUs should be explored to improve compute performance and save host CPUs. However, based on the characteristics of the accelerators, i.e., they achieve high throughput but can increase latency, what tasks to accelerate must be carefully selected.

REFERENCES

[1] Broadcom stingray smartnic accelerates baidu cloud services. https://www.broadcom.com/company/news/product-releases/53106, 2020.

[2] Amd pensando infrastructure accelerators. https://www.amd.com/en/accelerators/pensando, 2023.

[3] Intel infrastructure processing unit (intel ipu). https://www.intel.com/content/www/us/en/products/details/network-io/ipu.html, 2023.

[4] Kalray mppa dpu manycore. https://www.kalrayinc.com/products/mppa-technology/, 2023.

[5] Marvell octeon data processing units (dpus). https://www.marvell.com/products/data-processing-units.html, 2023.

[6] Transform the data center with nvidia dpus. https://www.nvidia.com/en-us/networking/products/data-processing-unit/, 2023.

[7] Alibaba Cloud Community. A detailed explanation about alibaba cloud cipu. https://www.alibabacloud.com/blog/599183, 2022.

[8] All Things Distributed. Reinventing virtualization with the aws nitro system. https://www.allthingsdistributed.com/2020/09/reinventing-virtualization-with-nitro.html, 2020.

[9] P. Antonopoulos, A. Budovski, C. Diaconu, A. H. Saenz, J. Hu, H. Kodavalla, D. Kossmann, S. Lingam, U. F. Minhas, N. Prakash, V. Purohit, H. Qu, C. S. Ravella, K. Reisteter, S. Shrotri, D. Tang, and V. Wakade. Socrates: The new SQL server in the cloud. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, pages 1743–1756. ACM, 2019.

[10] Arm. High-performance hardware support for floating-point operations. https://www.arm.com/technologies/floating-point.

[11] AWS Whitepaper. The components of the nitro system. https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/the-components-of-the-nitro-system.html, 2024.

[12] D. Baranchuk, A. Babenko, and Y. Malkov. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 202–216, 2018.

[13] C. Barthels, S. Loesing, G. Alonso, and D. Kossmann. Rack-scale in-memory join processing using RDMA. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, pages 1463–1475. ACM, 2015.

[14] M. Bhat, J. Crawford, R. Morin, and K. Shiv. Performance characterization of decimal arithmetic in commercial java workloads. In *2007 IEEE International Symposium on Performance Analysis of Systems & Software*, pages 54–61. IEEE, 2007.

[15] C. Binnig, A. Crotty, A. Galakatos, T. Kraska, and E. Zamanian. The end of slow networks: It's time for a redesign. *Proc. VLDB Endow.*, 9(7):528–539, 2016.

[16] W. Cao, Z. Liu, P. Wang, S. Chen, C. Zhu, S. Zheng, Y. Wang, and G. Ma. Polarfs: An ultra-low latency and failure resilient distributed file system for shared storage cloud database. *Proc. VLDB Endow.*, 11(12):1849–1862, 2018.

[17] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du. Characterizing, modeling, and benchmarking rocksdb key-value workloads at facebook. In *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, pages 209–223. USENIX Association, 2020.

[18] X. Chen, L. Yu, V. Liu, and Q. Zhang. Cowbird: Freeing cpus to compute by offloading the disaggregation of memory. In *Proceedings of the ACM SIGCOMM 2023 Conference, ACM SIGCOMM 2023, New York, NY, USA, 10-14 September 2023*, pages 1060–1073. ACM, 2023.

[19] X. Chen, J. Zhang, T. Fu, Y. Shen, S. Ma, K. Qian, L. Zhu, C. Shi, Y. Zhang, M. Liu, and Z. Wang. Demystifying datapath accelerator enhanced off-path smartnic. *CoRR*, abs/2402.03041, 2024.

[20] M. Chiosa, F. Maschi, I. Müller, G. Alonso, and N. May. Hardware acceleration of compression and encryption in SAP HANA. *Proc. VLDB Endow.*, 15(12):3277–3291, 2022.

[21] A. Depoutovitch, C. Chen, J. Chen, P. Larson, S. Lin, J. Ng, W. Cui, Q. Liu, W. Huang, Y. Xiao, and Y. He. Taurus Database: How to be Fast, Available, and Frugal in the Cloud. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*, pages 1463–1478, 2020.

[22] P. Deutsch. Rfc1951: Deflate compressed data format specification version 1.3, 1996.

[23] DuckDB Foundation. Duckdb – an in-process sql olap database management system. https://duckdb.org.

[24] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. S. III, S. Sosothikul, D. Terry, and A. Vig. Amazon dynamodb: A scalable, predictably performant, and fully managed nosql database service. In *Proceedings of the 2022 USENIX Annual Technical Conference, USENIX ATC 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 1037–1048. USENIX Association, 2022.

[25] D. Firestone, A. Putnam, S. Mundkur, D. Chiou, A. Dabagh, M. Andrewartha, H. Angepat, V. Bhanu, A. M. Caulfield, E. S. Chung, H. K. Chandrappa, S. Chaturmohta, M. Humphrey, J. Lavier, N. Lam, F. Liu, K. Ovtcharov, J. Padhye, G. Popuri, S. Raindel, T. Sapre, M. Shaw, G. Silva, M. Sivakumar, N. Srivastava, A. Verma, Q. Zuhair, D. Bansal, D. Burger, K. Vaid, D. A. Maltz, and A. G. Greenberg. Azure accelerated networking: Smartnics in the public cloud. In *15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018, Renton, WA, USA, April 9-11, 2018*, pages 51–66. USENIX Association, 2018.

[26] P. X. Gao, A. Narayan, S. Karandikar, J. Carreira, S. Han, R. Agarwal, S. Ratnasamy, and S. Shenker. Network requirements for resource disaggregation. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, pages 249–264. USENIX Association, 2016.

[27] Girish Bablani. Microsoft announces acquisition of fungible to accelerate datacenter innovation. https://blogs.microsoft.com/blog/2023/01/09/microsoft-announces-acquisition-of-fungible-to-accelerate-datacenter-innovation/, 2023.

[28] Google Cloud. AlloyDB for PostgreSQL under the hood: Intelligent, database-aware storage. https://cloud.google.com/blog/products/databases/alloydb-for-postgresql-intelligent-scalable-storage, 2022.

[29] S. Grant, A. Yelam, M. Bland, and A. C. Snoeren. Smartnic performance isolation with fairnic: Programmable networking for the cloud. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '20, page 681–693, New York, NY, USA, 2020. Association for Computing Machinery.

[30] Z. Guo, H. Zhang, C. Zhao, Y. Bai, M. M. Swift, and M. Liu. LEED: A low-power, fast persistent key-value store on smartnic jbofs. In H. Schulzrinne, V. Misra, E. Kohler, and D. A. Maltz, editors, *Proceedings of the ACM SIGCOMM 2023 Conference, ACM SIGCOMM 2023, New York, NY, USA, 10-14 September 2023*, pages 1012–1027. ACM, 2023.

[31] A. Gupta, A. Bansal, and V. Khanduja. Modern lossless compression techniques: Review, comparison and analysis. In *2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, pages 1–8. IEEE, 2017.

[32] G. Haas, M. Haubenschild, and V. Leis. Exploiting directly-attached nvme arrays in DBMS. In *10th Conference on Innovative Data Systems Research, CIDR 2020, Amsterdam, The Netherlands, January 12-15, 2020, Online Proceedings*. www.cidrdb.org, 2020.

[33] J. Hu, P. A. Bernstein, J. Li, and Q. Zhang. DPDPU: data processing with dpus. *CoRR*, abs/2407.13658, 2024.

[34] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems*, 32, 2019.

[35] J. Kim, I. Jang, W. Reda, J. Im, M. Canini, D. Kostić, Y. Kwon, S. Peter, and E. Witchel. Linefs: Efficient smartnic offload of a distributed file system with pipeline parallelism. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 756–771, New York, NY, USA, 2021. Association for Computing Machinery.

[36] D. Korolija, D. Koutsoukos, K. Keeton, K. Taranov, D. S. Milojicic, and G. Alonso. Farview: Disaggregated memory with operator off-loading for database engines. In *12th Conference on Innovative Data Systems Research, CIDR 2022, Chaminade, CA, USA, January 9-12, 2022*. www.cidrdb.org, 2022.

[37] S. Legtchenko, H. Williams, K. Razavi, A. Donnelly, R. Black, A. Douglas, N. Cheriere, D. Fryer, K. Mast, A. D. Brown, A. Klimovic, A. Slowey, and A. I. T. Rowstron. Understanding rack-scale disaggregated storage. In *9th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage 2017, Santa Clara, CA, USA, July 10-11, 2017*. USENIX Association, 2017.

[38] Y. Li, A. Kashyap, Y. Guo, and X. Lu. Compression analysis for bluefield-2/-3 data processing units: Lossy and lossless perspectives. *IEEE Micro*, 44(2):8–19, 2024.

[39] J. Lin, T. Ji, X. Hao, H. Cha, Y. Le, X. Yu, and A. Akella. Towards accelerating data intensive application's shuffle process using smartnics. In E. Smirni, K. Avrachenkov, P. Gill, and B. Urgaonkar, editors, *Abstract Proceedings of the 2023 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS 2023, Orlando, FL, USA, June 19-23, 2023*, pages 9–10. ACM, 2023.

[40] J. Liu, C. Maltzahn, C. D. Ulmer, and M. L. Curry. Performance characteristics of the bluefield-2 smartnic. *CoRR*, abs/2105.06619, 2021.

[41] M. Liu, T. Cui, H. Schuh, A. Krishnamurthy, S. Peter, and K. Gupta. Offloading distributed applications onto smartnics using ipipe. In *Proceedings of the ACM Special Interest Group on Data Communication*, SIGCOMM '19, page 318–333, New York, NY, USA, 2019. Association for Computing Machinery.

[42] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[43] Marvell. Marvell octeon sdk. https://www.marvell.com/content/dam/marvell/en/public-collateral/embedded-processors/marvell-octeon-tx2-sdk-solutions-brief.pdf, 2020.

[44] B. Michalowicz, K. K. Suresh, H. Subramoni, D. K. Panda, and S. Poole. Dpu-bench: A micro-benchmark suite to measure offload efficiency of smartnics. In *Practice and Experience in Advanced Research Computing, PEARC 2023, Portland, OR, USA, July 23-27, 2023*, pages 94–101. ACM, 2023.

[45] B. Michalowicz, K. K. Suresh, H. Subramoni, D. K. D. K. Panda, and S. W. Poole. Battle of the bluefields: An in-depth comparison of the bluefield-2 and bluefield-3 smartnics. In *IEEE Symposium on High-Performance Interconnects, HOTI 2023, Virtual Conference, USA, August 23-25, 2023*, pages 41–48. IEEE, 2023.

[46] Microsoft. Azure cache for redis: Distributed, in-memory, scalable solution providing super-fast data access. https://azure.microsoft.com/en-us/products/cache, 2024.

[47] Y. Moon, S. Lee, M. A. Jamshed, and K. Park. AccelTCP: Accelerating network applications with stateful TCP offloading. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 77–92, Santa Clara, CA, Feb. 2020. USENIX Association.

[48] D. Morera. The rise of dpus in the infrastructure. https://blogs.vmware.com/cloud-foundation/2024/07/16/the-rise-of-dpus-in-the-infrastructure/, 2024.

[49] NVIDIA. Nvidia connectx-7 nic. https://resources.nvidia.com/en-us-accelerated-networking-resource-library/connectx-7-datasheet.

[50] NVIDIA Developer. Nvidia doca software framework. https://developer.nvidia.com/networking/doca, 2023.

[51] NVMe Express. Nvme over fabrics. https://nvmexpress.org/wp-content/uploads/NVMe_Over_Fabrics.pdf, 2016.

[52] K. Pacella. Accelerating microsoft azure with amd dpus. https://community.amd.com/t5/corporate/accelerating-microsoft-azure-with-amd-dpus/ba-p/604170, 2023.

[53] S. J. Park, R. Govindan, K. Shen, D. E. Culler, F. Özcan, G. Kim, and H. Levy. Lovelock: Towards smart nic-hosted clusters. *CoRR*, abs/2309.12665, 2023.

[54] P. M. Phothilimthana, M. Liu, A. Kaufmann, S. Peter, R. Bodik, and T. Anderson. Floem: A programming system for NIC-Accelerated network applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 663–679, Carlsbad, CA, Oct. 2018. USENIX Association.

[55] H. N. Schuh, W. Liang, M. Liu, J. Nelson, and A. Krishnamurthy. Xenic: Smartnic-accelerated distributed transactions. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 740–755, New York, NY, USA, 2021. Association for Computing Machinery.

[56] R. Shashidhara, T. Stamler, A. Kaufmann, and S. Peter. FlexTOE: Flexible TCP offload with Fine-Grained parallelism. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 87–102, Renton, WA, Apr. 2022. USENIX Association.

[57] S. Sivasubramanian. Amazon dynamodb: a seamlessly scalable non-relational database service. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20-24, 2012*, pages 729–730. ACM, 2012.

[58] Symas. Embedded database lmdb: Lightning memory-mapped database. https://www.symas.com/mdb, 2025.

[59] L. Thostrup, G. Doci, N. Boeschen, M. Luthra, and C. Binnig. Distributed GPU joins on fast rdma-capable networks. *Proc. ACM Manag. Data*, 1(1):29:1–29:26, 2023.

[60] L. Thostrup, D. Failing, T. Ziegler, and C. Binnig. A dbms-centric evaluation of bluefield dpus on fast networks. In *International Workshop on Accelerating Analytics and Data Management Systems Using Modern Processor and Storage Architectures, ADMS@VLDB 2022, Sydney, Australia, September 5, 2022*, pages 1–10, 2022.

[61] A. Verbitski, A. Gupta, D. Saha, M. Brahmadesam, K. Gupta, R. Mittal, S. Krishnamurthy, S. Maurice, T. Kharatishvili, and X. Bao. Amazon aurora: Design considerations for high throughput cloud-native relational databases. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, pages 1041–1052. ACM, 2017.

[62] M. Vuppalapati, J. Miron, R. Agarwal, D. Truong, A. Motivala, and T. Cruanes. Building An Elastic Query Engine on Disaggregated Storage. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 449–462, 2020.

[63] J. Wang and Q. Zhang. Disaggregated database systems. In S. Das, I. Pandis, K. S. Candan, and S. Amer-Yahia, editors, *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, Seattle, WA, USA, June 18-23, 2023*, pages 37–44. ACM, 2023.

[64] Z. Wang, C. Wang, and L. Wang. Dpubench: An application-driven scalable benchmark suite for comprehensive dpu evaluation. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 3(2):100120, 2023.

[65] X. Wei, R. Chen, Y. Yang, R. Chen, and H. Chen. Characterizing off-path smartnic for accelerating distributed systems. In *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2023, Boston, MA, USA, July 10-12, 2023*, pages 987–1004. USENIX Association, 2023.

[66] I. Yu. Alibaba cloud unveils new cloud infrastructure to power data centers. https://www.alizila.com/alibaba-cloud-unveils-new-cloud-infrastructure-to-power-data-centers/, 2022.

[67] Q. Zhang, P. A. Bernstein, D. S. Berger, and B. Chandramouli. Redy: Remote dynamic memory cache. *Proc. VLDB Endow.*, 15(4):766–779, 2021.

[68] Q. Zhang, P. A. Bernstein, B. Chandramouli, J. Hu, and Y. Zheng. DDS: dpu-optimized disaggregated storage. *Proc. VLDB Endow.*, 17(11):3304–3317, 2024.

[69] Q. Zhang, Y. Cai, S. Angel, V. Liu, A. Chen, and B. T. Loo. Rethinking Data Management Systems for Disaggregated Data Centers. In *Conference on Innovative Data Systems Research (CIDR)*, 2020.

[70] Q. Zhang, Y. Cai, X. Chen, S. Angel, A. Chen, V. Liu, and B. T. Loo. Understanding the Effect of Data Center Resource Disaggregation on Production DBMSs. *Proceedings of the VLDB Endowment (PVLDB)*, 13(9):1568–1581, 2020.

[71] Z. Zhou, Y. Li, S. Johnson, N. Finamore, C. Asto, R. Cyphers, and C. Delimitrou. Microservice benchmarking on intel ipus running napatech software. https://people.csail.mit.edu/delimitrou/papers/2022.intel.ipu.pdf, 2022.